

CSA43 - Assessment Tool 1 Answers

COURSE NAME: Internet Programming **COURSE CODE:** CSA43

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool: Scenario-Based Assessment **Weightage:** 40% **Total Marks:** 30

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Question 1 - Servlet-Based Student Registration System (10 Marks)

Scenario: A college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.

1.1 Understanding the Scenario

A student fills an HTML registration form (name, register number, department, email, phone, etc.) in the browser. On submission, the browser sends this data to a server-side Java program — a **Servlet** — deployed inside a **Web Container** (e.g., Apache Tomcat). The servlet reads the submitted values, validates/processes them (for example, stores them in a database or a session, checks for duplicate registration), and sends an HTML response back confirming successful registration or reporting an error.

1.2 Architecture of a Servlet-Based Application

A servlet-based web application follows a layered client-server architecture:

1. **Client (Browser)** – Sends an HTTP request (GET/POST) generated when the student submits the registration form.
2. **Web Server / Web Container** – Software such as Apache Tomcat that implements the Servlet API. It is responsible for:
 - Loading and instantiating the servlet class.
 - Creating `HttpServletRequest` and `HttpServletResponse` objects for every request.
 - Managing the servlet life cycle (`init()`, `service()`, `destroy()`).
 - Multithreading – one thread per request, but only a single servlet instance.
3. **Servlet** – The Java class (extending `HttpServlet`) that contains the business logic to process the registration data.
4. **Database (optional but typical)** – Accessed through JDBC to persist student records permanently.
5. **Response** – The servlet writes an HTML/JSON response through the `PrintWriter` obtained from the response object, which the container sends back to the browser.

Flow: Browser --(HTTP Request)--> Web Container --> Servlet (business logic) --> Database --> Servlet builds response --> Web Container --(HTTP Response)--> Browser

The Web Container acts as the mediator — the servlet never talks to the raw socket; it only works with the request/response abstractions that the container provides, which makes servlets portable and container-independent.

1.3 Registration Form (HTML)

```
<!DOCTYPE html>
<html>
<head><title>Student Registration</title></head>
<body>
  <h2>Student Registration Form</h2>
  <form action="RegisterServlet" method="post">
    Name: <input type="text" name="sname" required/><br/>
    Register No: <input type="text" name="regno" required/><br/>
    Department: <input type="text" name="dept" required/><br/>
    Email: <input type="email" name="email" required/><br/>
    <input type="submit" value="Register"/>
  </form>
</body>
</html>
```

1.4 Servlet Implementation - Handling GET and POST

```
import java.io.*;
import java.sql.*;
import javax.servlet.*;
import javax.servlet.http.*;
import javax.servlet.annotation.WebServlet;

@WebServlet("/RegisterServlet")
public class RegisterServlet extends HttpServlet {

    // Called once when the servlet is first loaded
    @Override
    public void init() throws ServletException {
        System.out.println("RegisterServlet initialized - resources set up here");
        // e.g., initialize a DataSource / connection pool once
    }

    // Handles GET requests - e.g., displaying the form or a status page
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<h3>Please use the registration form to submit your details (POST).
</h3>");
    }

    // Handles POST requests - actual form submission with sensitive data
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        // Step 1: Read form data
        String name = request.getParameter("sname");
        String regNo = request.getParameter("regno");
        String dept = request.getParameter("dept");
        String email = request.getParameter("email");

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Step 2: Basic validation
        if (name == null || name.trim().isEmpty() || regNo == null || regNo.trim().isEmpty())
        {
            out.println("<h3>Error: Name and Register Number are mandatory.</h3>");
            return;
        }

        // Step 3: Persist to database (see Question 3 for full JDBC pattern)
        try (Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/college_db", "root", "password")) {
```

```

String sql = "INSERT INTO student(name, regno, dept, email) VALUES (?, ?, ?, ?)";
PreparedStatement ps = con.prepareStatement(sql);
ps.setString(1, name);
ps.setString(2, regNo);
ps.setString(3, dept);
ps.setString(4, email);
ps.executeUpdate();

out.println("<h3>Registration Successful for " + name + "</h3>");

} catch (SQLException e) {
    out.println("<h3>Registration failed due to a server error.</h3>");
    e.printStackTrace();
}

}

// Called once when the servlet is being taken out of service
@Override
public void destroy() {
    System.out.println("RegisterServlet destroyed - resources released here");
}
}

```

1.5 The Servlet Life Cycle

Every servlet in the container passes through three life-cycle stages, managed entirely by the Web Container, not by the programmer:

Phase	Method	When it runs	Purpose
Loading & Instantiation	Constructor + <code>init(ServletConfig)</code>	Exactly once, the first time the servlet is requested (or at container start-up if <code>load-on-startup</code> is set)	One-time setup: open a DB connection pool, read configuration parameters, load reference data
Request Handling	<code>service()</code> (which internally dispatches to <code>doGet()</code> , <code>doPost()</code> , <code>doPut()</code> , <code>doDelete()</code> based on the HTTP method)	Once per client request , on its own thread	Executes the actual business logic for that particular request
Destruction	<code>destroy()</code>	Exactly once, when the container shuts down the servlet (server shutdown, redeployment, or explicit unloading)	Release resources: close connections, flush logs, save state

Key characteristic: The container creates only **one instance** of the servlet class; `service()` (and hence `doGet/doPost`) is invoked concurrently on **multiple threads**, one per incoming request. This is why servlets should avoid storing per-request data in instance variables (thread-safety concern) — all request-specific data must live in local variables inside `doGet/doPost` or in the `HttpServletRequest/HttpSession` objects.

1.6 GET vs POST - Which is Better for Sensitive Data?

Aspect	GET	POST
Data location	Appended to URL as query string	Sent in the HTTP request body
Visibility	Visible in browser history, server logs, bookmarks	Not shown in the URL
Data length	Limited by URL length restrictions	Practically unlimited (large forms, files)
Caching	Can be cached/bookmarked	Not cached by default
Idempotency	Safe, idempotent (should not change server state)	Used for state-changing operations

Justification: For the student registration scenario, **POST is the appropriate method for handling sensitive data** (name, register number, personal email, etc.) because: 1. The data travels in the request body rather than the URL, so it is not exposed in browser history, server access logs, or shared/bookmarked links. 2. POST has no practical size limitation, accommodating all registration fields. 3. POST is semantically correct here since registration **creates/changes state** on the server (a new database record), whereas GET is meant for safe, idempotent, read-only operations. 4. Combined with HTTPS, POST prevents sensitive data from being transmitted in plaintext as part of the URL.

`init()` and `destroy()` run only once and are not involved in per-request data handling, so the real choice that matters for sensitive data is between `doGet()` and `doPost()` — and `doPost()` is the correct, more secure choice.

Question 2 - Session Management with HttpSession and Cookies (10 Marks)

Scenario: A website requires users to log in and maintain their session across multiple pages after authentication.

2.1 Understanding the Scenario

HTTP is a **stateless** protocol — each request/response pair is independent, and the server does not, by default, remember a client between requests. To let a user “stay logged in” while navigating multiple pages, the application must implement **session tracking**, so the server can recognize that a sequence of requests belongs to the same authenticated user.

2.2 Design: Login + Session Creation

Login Servlet (creates the session on successful authentication):

```
@WebServlet("/LoginServlet")
public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String username = request.getParameter("username");
        String password = request.getParameter("password");

        PrintWriter out = response.getWriter();

        if (authenticate(username, password)) { // checks against DB
            // Step 1: Create (or retrieve) a session for this user
            HttpSession session = request.getSession(true);
            session.setAttribute("user", username);
            session.setMaxInactiveInterval(30 * 60); // 30-minute timeout

            // Step 2: Optionally set a persistent cookie e.g. "remember me"
            Cookie rememberCookie = new Cookie("lastUser", username);
            rememberCookie.setMaxAge(7 * 24 * 60 * 60); // 7 days
            rememberCookie.setHttpOnly(true);
            response.addCookie(rememberCookie);

            response.sendRedirect("dashboard.jsp");
        } else {
            out.println("<h3>Invalid username or password</h3>");
        }
    }

    private boolean authenticate(String u, String p) {
        // JDBC lookup against the users table (omitted for brevity)
        return u != null && p != null;
    }
}
```

Protected page / filter that checks the session on every subsequent request:

```
@WebServlet("/DashboardServlet")
public class DashboardServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        // Step 3: Validate session on every protected request
        HttpSession session = request.getSession(false); // don't create a new one
        PrintWriter out = response.getWriter();

        if (session == null || session.getAttribute("user") == null) {
            response.sendRedirect("login.jsp"); // not logged in / session expired
            return;
        }

        String user = (String) session.getAttribute("user");
        out.println("<h3>Welcome back, " + user + "!</h3>");
    }
}
```

Logout Servlet (destroys the session):

```
@WebServlet("/LogoutServlet")
public class LogoutServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        HttpSession session = request.getSession(false);
        if (session != null) session.invalidate();
        response.sendRedirect("login.jsp");
    }
}
```

2.3 How Session Tracking Works

1. On the very first request (login), the servlet calls `request.getSession(true)`. The container generates a **unique Session ID** and creates a server-side `HttpSession` object that can hold key-value attributes (e.g., `user`, `role`, `cartItems`).
2. The container sends the Session ID back to the browser, by default inside a special cookie called **JSESSIONID** (an HTTP response header `Set-Cookie: JSESSIONID=...`).
3. On every subsequent request, the browser automatically attaches the `JSESSIONID` cookie back to the server.
4. The container reads the incoming `JSESSIONID`, looks up the matching `HttpSession` object in server memory, and hands it to the servlet through `request.getSession()`.
5. The servlet can then retrieve attributes stored earlier (e.g., `session.getAttribute("user")`) without the user having to log in again on every page.
6. The session ends when: the user logs out (`session.invalidate()`), the configured timeout elapses with no activity (`setMaxInactiveInterval()`), or the server restarts (unless persisted).
7. If cookies are disabled in the browser, the container can fall back to **URL rewriting**, appending the session ID to every link (`;jsessionid=...`) so tracking still works.

2.4 Session-Based Tracking vs Cookies - Comparison

Aspect	HttpSession (server-side)	Cookies (client-side)
Storage location	Data stored on the server ; only the session ID travels to the client	Entire data stored on the client's browser
Security	More secure - sensitive data never leaves the server	Less secure - data sits in the browser and can be read/tampered with unless encrypted
Capacity	Can store large/complex objects (no size limit beyond server)	Limited to ~4 KB per cookie, and a limited number per domain

	memory)	
Lifetime	Typically ends when the browser closes or after a configured timeout	Can be made to persist for days/months (<code>setMaxAge</code>) independent of browser closing
Dependency	Actually relies on a cookie (<code>JSESSIONID</code>) or URL rewriting under the hood to identify the client	Self-contained; doesn't need a server-side session store
Performance/Scalability	Consumes server memory per active user; needs session replication in clustered servers	Stateless from the server's point of view; scales more easily across multiple servers
Typical use	Login state, shopping cart, multi-step form data	"Remember me" username, user preferences, tracking/analytics IDs

Conclusion: In practice, robust systems use both together — a lightweight cookie (`JSESSIONID`) to *identify* the client, and a server-side `HttpSession` to *securely store* the actual authenticated state, giving the security of server-side storage with the convenience of automatic cookie transport.

Question 3 - JDBC-Based Product Management for an E-Commerce Platform (10 Marks)

Scenario: An e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

3.1 JDBC Architecture

JDBC (Java Database Connectivity) is an API that provides a uniform way for Java applications to interact with relational databases, regardless of the underlying vendor. Its architecture has two layers:

1. **JDBC API Layer** - Provides the standard interfaces/classes the programmer codes against: `DriverManager`, `Connection`, `Statement/PreparedStatement`, `ResultSet`, `SQLException`. This layer is database-independent.
2. **JDBC Driver Layer** - A vendor-specific implementation (e.g., MySQL Connector/J, Oracle's `ojdbc` driver) that translates the standard JDBC calls into the database's native protocol. Most modern drivers are **Type 4 (pure Java, direct-to-database) drivers**.

Core components:

- **DriverManager** - Manages the set of registered JDBC drivers and creates a `Connection` given a JDBC URL.
- **Connection** - Represents an active session with the specific database.
- **Statement / PreparedStatement** - Used to send SQL to the database. `PreparedStatement` is precompiled and parameterized, protecting against SQL injection.
- **ResultSet** - A cursor-like object holding the rows returned by a `SELECT` query.
- **SQLException** - The checked exception type used for all database access errors.

Flow: Java Application --> JDBC API --> JDBC Driver --> Database --> ResultSet/Update count --> back to Application

3.2 Steps in Database Connectivity

1. **Load/register the driver** - with modern JDBC 4.0+ drivers this happens automatically via `META-INF/services`, but can also be done with `Class.forName("com.mysql.cj.jdbc.Driver")`.
2. **Establish the connection** - `DriverManager.getConnection(url, user, password)`.
3. **Create a statement** - prefer `PreparedStatement` for parameterized, safe queries.
4. **Execute the query** - `executeUpdate()` for INSERT/UPDATE/DELETE, `executeQuery()` for SELECT.
5. **Process the results** - iterate the `ResultSet` with `rs.next()` and read columns.

6. **Close resources** – close `ResultSet`, `Statement`, and `Connection` (ideally via **try-with-resources**, which auto-closes them even if an exception occurs).

3.3 Servlet: Insert Product Data

```
@WebServlet("/AddProductServlet")
public class AddProductServlet extends HttpServlet {

    private static final String URL = "jdbc:mysql://localhost:3306/ecommerce_db";
    private static final String USER = "root";
    private static final String PASS = "password";

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String name = request.getParameter("pname");
        String price = request.getParameter("price");
        String qty = request.getParameter("qty");

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String sql = "INSERT INTO product(name, price, quantity) VALUES (?, ?, ?)";

        try (Connection con = DriverManager.getConnection(URL, USER, PASS);
            PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setString(1, name);
            ps.setDouble(2, Double.parseDouble(price));
            ps.setInt(3, Integer.parseInt(qty));

            int rows = ps.executeUpdate();
            out.println("<h3>" + rows + " product record inserted successfully.</h3>");

        } catch (SQLException e) {
            out.println("<h3>Database error while inserting product.</h3>");
            e.printStackTrace();
        } catch (NumberFormatException e) {
            out.println("<h3>Invalid price/quantity format.</h3>");
        }
    }
}
```

3.4 Servlet: Retrieve Product Data

```
@WebServlet("/ViewProductsServlet")
public class ViewProductsServlet extends HttpServlet {

    private static final String URL = "jdbc:mysql://localhost:3306/ecommerce_db";
    private static final String USER = "root";
    private static final String PASS = "password";

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<table border='1'><tr><th>ID</th><th>Name</th><th>Price</th><th>Qty</th>
</tr>");

        String sql = "SELECT id, name, price, quantity FROM product";

        try (Connection con = DriverManager.getConnection(URL, USER, PASS);
            Statement st = con.createStatement();
            ResultSet rs = st.executeQuery(sql)) {

            while (rs.next()) {
                out.println("<tr><td>" + rs.getInt("id") + "</td><td>"
                    + rs.getString("name") + "</td><td>"

```

```

        + rs.getDouble("price") + "</td><td>"
        + rs.getInt("quantity") + "</td></tr>");
    }

    } catch (SQLException e) {
        out.println("Error retrieving products: " + e.getMessage());
        e.printStackTrace();
    }

    out.println("</table>");
}
}

```

3.5 Exception Handling in JDBC

- All JDBC operations that touch the database throw the **checked exception** `SQLException`, forcing the developer to handle database failures explicitly (connection refused, constraint violations, syntax errors, etc.).
- **Try-with-resources** (used above) ensures `Connection`, `Statement`, and `ResultSet` objects are closed automatically in `finally`-equivalent fashion, preventing connection leaks even when an exception occurs mid-way.
- `SQLException` exposes useful diagnostic methods: `getMessage()`, `getSQLState()`, and `getErrorCode()`, which can be logged for debugging without exposing raw error details to the end user.
- Numeric parsing (`Double.parseDouble()`, `Integer.parseInt()`) can throw `NumberFormatException` for malformed user input; this should be caught separately from `SQLException` so form-validation errors are reported clearly instead of being mistaken for database errors.
- **PreparedStatement** additionally guards against **SQL Injection** — user input is bound as parameters rather than concatenated into the SQL string, so a malicious value like `' ; DROP TABLE product; --` is treated purely as data, never as executable SQL.
- In production, it is good practice to log the full stack trace server-side while returning a generic, user-friendly error message to the client, so internal schema/database details are never leaked.